

UNIT – 1
ASSESSMENT – 1
REPORT

NIHARIKA PREEJESH
192524506

1. Python Code for Water Jug Problem

```
# Water Jug Problem
```

```
def water_jug():
```

```
    states = [  
        "(0,0)",  
        "(0,3)",  
        "(3,0)",  
        "(3,3)",  
        "(4,2)",  
        "(0,2)",  
        "(2,0)"  
    ]
```

```
    actions = [  
        "Initial State",  
        "Fill 3-gallon Jug",  
        "Pour 3-gallon into 4-gallon",  
        "Fill 3-gallon Jug Again",  
        "Pour into 4-gallon until full",  
        "Empty 4-gallon Jug",  
        "Pour remaining water into 4-gallon"  
    ]
```

```
print("Water Jug Solution\n")

for action, state in zip(actions, states):
    print(action, "->", state)

print("\nGoal Achieved!")
print("Exactly 2 gallons are in the 4-gallon jug.")

water_jug()
```

2. Python Code for Mars Rover Problem

Mars Rover Agent Simulation

```
class MarsRover:
```

```
    def __init__(self):
        self.location = "Landing Site"
        self.samples = 0
```

```
    def move(self):
        print("Moving to a new location...")
```

```
    def capture_image(self):
        print("Capturing surface image...")
```

```
    def collect_sample(self):
        self.samples += 1
        print("Collecting soil sample...")
```

```

def analyze_sample(self):
    print("Analyzing collected sample...")

def send_data(self):
    print("Sending data to Earth...")

def display_status(self):
    print("\nMission Summary")
    print("-----")
    print("Location :", self.location)
    print("Samples Collected :", self.samples)

rover = MarsRover()

print("Mars Rover Mission Started\n")

rover.move()
rover.capture_image()
rover.collect_sample()
rover.analyze_sample()
rover.send_data()

rover.display_status()

```

3. Python Code for 8 Queens Problem:

8 Queens Problem using Backtracking

N = 8

```
board = [[0 for _ in range(N)] for _ in range(N)]
```

```
def is_safe(row, col):
```

```
    # Check left side
```

```
    for i in range(col):
```

```
        if board[row][i] == 1:
```

```
            return False
```

```
    # Upper diagonal
```

```
    i = row
```

```
    j = col
```

```
    while i >= 0 and j >= 0:
```

```
        if board[i][j] == 1:
```

```
            return False
```

```
        i -= 1
```

```
        j -= 1
```

```
    # Lower diagonal
```

```
    i = row
```

```
    j = col
```

```
    while i < N and j >= 0:
```

```
        if board[i][j] == 1:
```

```
            return False
```

```
        i += 1
```

```
        j -= 1
```

```
    return True
```

```
def solve(col):
```

```
    if col >= N:
```

```
        return True
```

```
    for row in range(N):
```

```
        if is_safe(row, col):
```

```
            board[row][col] = 1
```

```
            if solve(col + 1):
```

```
                return True
```

```
            board[row][col] = 0
```

```
    return False
```

```
if solve(0):
```

```
    print("Solution Found:\n")
```

```
    for row in board:
```

```
        print(row)
```

```
else:
```

```
    print("No Solution Exists")
```

4. Python Code for OLA cab booking Problem

```
# OLA Cab Booking Simulation

pickup = input("Enter Pickup Location: ")
destination = input("Enter Destination: ")

print("\nAvailable Cab Types")
print("1. Mini")
print("2. Micro")
print("3. Sedan")
print("4. Shared")
print("5. Prime")

choice = input("\nSelect Cab Type: ")

cab_types = {
    "1": "Mini",
    "2": "Micro",
    "3": "Sedan",
    "4": "Shared",
    "5": "Prime"
}

if choice in cab_types:
    print("\nCab Booked Successfully!")
    print("Cab Type :", cab_types[choice])
    print("Pickup :", pickup)
    print("Destination :", destination)
    print("Driver Assigned")
    print("Trip Started...")
    print("Trip Completed Successfully!")
```

```
else:  
    print("Invalid Cab Selection")
```

5. Python Code for Uniform Cost Search Problem:

```
# Uniform Cost Search (UCS)
```

```
from queue import PriorityQueue
```

```
graph = {  
    'S': [('A', 1), ('G', 12)],  
    'A': [('B', 3), ('C', 1)],  
    'B': [('G', 3)],  
    'C': [('D', 1)],  
    'D': [('G', 3)],  
    'G': []  
}
```

```
def uniform_cost_search(start, goal):
```

```
    pq = PriorityQueue()  
    pq.put((0, start, [start]))
```

```
    visited = set()
```

```
    while not pq.empty():
```

```
        cost, node, path = pq.get()
```

```
if node == goal:  
    return cost, path
```

```
if node not in visited:
```

```
    visited.add(node)
```

```
    for neighbour, edge_cost in graph[node]:  
        if neighbour not in visited:  
            pq.put((cost + edge_cost,  
                    neighbour,  
                    path + [neighbour]))
```

```
    return None
```

```
cost, path = uniform_cost_search('S', 'G')
```

```
print("Least Cost Path :", " -> ".join(path))
```

```
print("Total Cost :", cost)
```